MALLA REDDY COLLEGE OF ENGINEERING AND TECHNOLOGY

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

HireLens AI: Explainable AI Resume Screening System

Prepared by

S.No	Roll Number	Student Name
1.	22N31A6627	Bheemanapally Abhaya Sri
2.	22N31A6629	Bochkar Nikhith
3.	22N31A6666	Jakkireddy Sri Charan Reddy

Supervisor:	Mr. S. Venkateswara Raju
Designation:	Assistant Professor
Department:	Computational Intelligence
Batch ID:	
Date:	
Supervisor Sign. & Date	

Department of Computational Intelligence

Title of the Project: HireLens AI: Explainable AI Resume Screening System

Content

CONTENTS ..... **II**

REVISIONS ..... **II**

1 INTRODUCTION **1**

1.1 DOCUMENT PURPOSE 1
1.2 PROJECT SCOPE 1
1.3 EXISTING SYSTEMS 1
1.4 PROBLEMS WITH EXISTING SYSTEMS 2
1.5 PROPOSED SYSTEMS 2
1.6 ADVANTAGES OF PROPOSED SYSTEMS 2

2 OVERALL DESCRIPTION **3**

2.1 FEASIBILITY STUDY 3
2.2 PRODUCT FUNCTIONALITY 3
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS 3
2.4 ASSUMPTIONS AND DEPENDENCIES 3

3 FUNCTIONAL REQUIREMENTS **4**

3.1 SOFTWARE REQUIREMENT SPECIFICATIONS 4
3.2 HARDWARE REQUIREMENTS SPECIFICATIONS 4
3.3 **UML DIAGRAMS** 5

4 OTHER NON-FUNCTIONAL REQUIREMENTS **9**

4.1 PERFORMANCE REQUIREMENTS 9
4.2 SAFETY AND SECURITY REQUIREMENTS 9
4.3 SOFTWARE QUALITY ATTRIBUTES 9

5 OTHER REQUIREMENTS **10**

5.1 DATABASE REQUIREMENTS 10
5.2 INTERNATIONALIZATION REQUIREMENTS 10
5.3 LEGAL REQUIREMENTS 10
5.4 REUSE OBJECTIVES 10
5.5 DEVELOPMENT ENVIRONMENT REQUIREMENTS 10
5.6 DOCUMENTATION REQUIREMENTS 10

6 REFERENCES **11**

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	B. Abhaya sri	Primary Revision giving an overall view of the project and document.	

1 Introduction

HireLens AI is a web-based, full-stack software system designed to automate and explain the resume screening process for recruiters and hiring managers. It accepts resumes in PDF, DOCX, and TXT formats alongside job description text, processes both through a modular Explainable AI (XAI) pipeline, and produces a match score with a transparent, human-readable justification for every screening decision. Built on a three-tier architecture — React 19 frontend, FastAPI backend, and SQLite database via RESTful API — the system combines automated information extraction, weighted multi-dimensional scoring, and natural language explainability to address both screening efficiency and decision transparency. This document formally captures all functional and non-functional requirements of HireLens AI and serves as the authoritative reference for developers, evaluators, and academic assessors.

1.1 Document Purpose

This SRS defines the requirements of HireLens AI — an explainable AI resume screening system. It guides developers, testers, and evaluators on the system's functionality, design constraints, and scope.

1.2 Project/Product Scope

- Web-based resume screening with PDF, DOCX, and TXT support.
- AI scoring across four dimensions: skills (40%), experience (25%), projects (20%), education (15%).
- Binary prediction (Selected / Rejected) with a human-readable explanation per candidate.
- Batch processing, ranked leaderboard, and side-by-side candidate comparison.

1.3 Existing System

- Commercial ATS (Workday, Greenhouse) — enterprise-grade but black-box; no decision explanation.
- Keyword-Based Filters — eliminate qualified candidates who use synonymous skill terms.
- LinkedIn Recruiter AI — proprietary matching with no explanation for candidate rankings.
- Open-Source ATS — basic pipeline tools with no AI scoring or explainability.

1.4 Problems with Existing System

- No explanation provided for hiring decisions — recruiters cannot audit results.
- Keyword fragility: synonymous terms (e.g., "ML" vs. "machine learning") cause false rejections.
- Single relevance score — no breakdown by skills, experience, education, or projects.
- Enterprise tools are costly and inaccessible to small teams and academic institutions.

1.5 Proposed system

- HireLens AI: six-stage pipeline — text extraction, parsing, job analysis, scoring, aggregation, and explainability.
- Weighted Linear Combination model with per-dimension scores communicated clearly to the recruiter.
- Explainability module: feature contributions, qualitative labels, matched/missing skills, and strengths/weaknesses.
- Fully integrated React 19 + FastAPI + SQLite web app, deployable locally without cloud dependencies.

1.6 Advantages of Proposed Systems

- SFull transparency: every decision includes a feature breakdown and natural language summary.
- Multi-format support: PDF, DOCX, and TXT processed in a single unified pipeline.
- Zero cost: built entirely on open-source libraries — no paid APIs or cloud subscriptions needed.
- Modular design: each AI module is independently upgradeable without affecting other components.

2 Overall Description

2.1 Feasibility Study

Technical Feasibility:

- Python 3.10+ with FastAPI and SQLAlchemy; no GPU required — all scoring runs on a standard CPU.

Economic Feasibility:

- Entire stack uses open-source libraries (MIT, Apache 2.0) at zero cost — no paid APIs needed.

Operational Feasibility:

- Three-step guided UI requires no training; AI outputs shown as natural language and visual gauges.

Schedule Feasibility:

- Modular architecture allowed parallel development of parsing, scoring, and explainability modules.

2.2 Product Functionality

- Resume Upload & Parsing: Accepts PDF, DOCX, TXT (≤ 10 MB); extracts skills, education, experience, projects, and certifications.
- Job Matching: Computes match score (0–100%), predicts Selected/Rejected, and generates explanation with matched/missing skills.
- Batch Processing: Matches all uploaded resumes against one job in a single call; returns a ranked leaderboard.

2.3 Design and Implementation Constraints

- Backend: Python 3.10+, FastAPI 0.115.0, SQLAlchemy 2.0.35; Frontend: React 19, Vite 8.0.0.
- Resume uploads limited to .pdf, .docx, .txt extensions with a 10 MB size cap.
- SQLite single-writer constraint; no concurrent multi-user write operations supported.

2.4 Assumptions and Dependencies

- Job descriptions are written in English with standard recruitment phrasing.
- Python 3.10+, Node.js 18+, and npm are installed in the development environment.
- Internet connection required on first run to download sentence-transformer model weights.

3 Functional Requirements

3.1 Software Requirement Specifications

Backend:

- Python 3.10+, FastAPI 0.115.0, Uvicorn 0.30.0, SQLAlchemy 2.0.35
- pdfplumber 0.11.4 (PDF parsing), python-docx 1.1.2 (DOCX parsing)
- spaCy 3.7.6, sentence-transformers 3.1.0, scikit-learn 1.5.2, SHAP 0.46.0, LIME 0.2.0.1

Frontend:

- Node.js 18+, React 19.2.4, React Router DOM 7.13.1, Vite 8.0.0
- TailwindCSS 4.2.1, Axios 1.13.6, Chart.js 4.5.1, react-dropzone 15.0.0

3.2 Hardware Requirements Specifications

Component	Minimum	Recommended
Processor	Intel Core i5 (8th gen) / AMD Ryzen 5 3000, 1.6 GHz	Intel Core i7 (10th gen+) / AMD Ryzen 7, 2.4 GHz+
Memory (RAM)	8 GB minimum	16 GB recommended
Storage	5 GB free disk space	SSD with 10 GB free space
Operating System	Windows 10 (64-bit), macOS 12.0, or Ubuntu 20.04 LTS	Latest version of above OS
Network	Active connection for initial setup	Stable broadband connection
Display	1280×720 pixels minimum	1920×1080 Full HD
Browser	Chrome 110+, Firefox 110+, Edge 110+	Latest Chrome or Edge (Chromium-based)

3.3 UML Diagrams

3.3.1 Usecase Diagram:

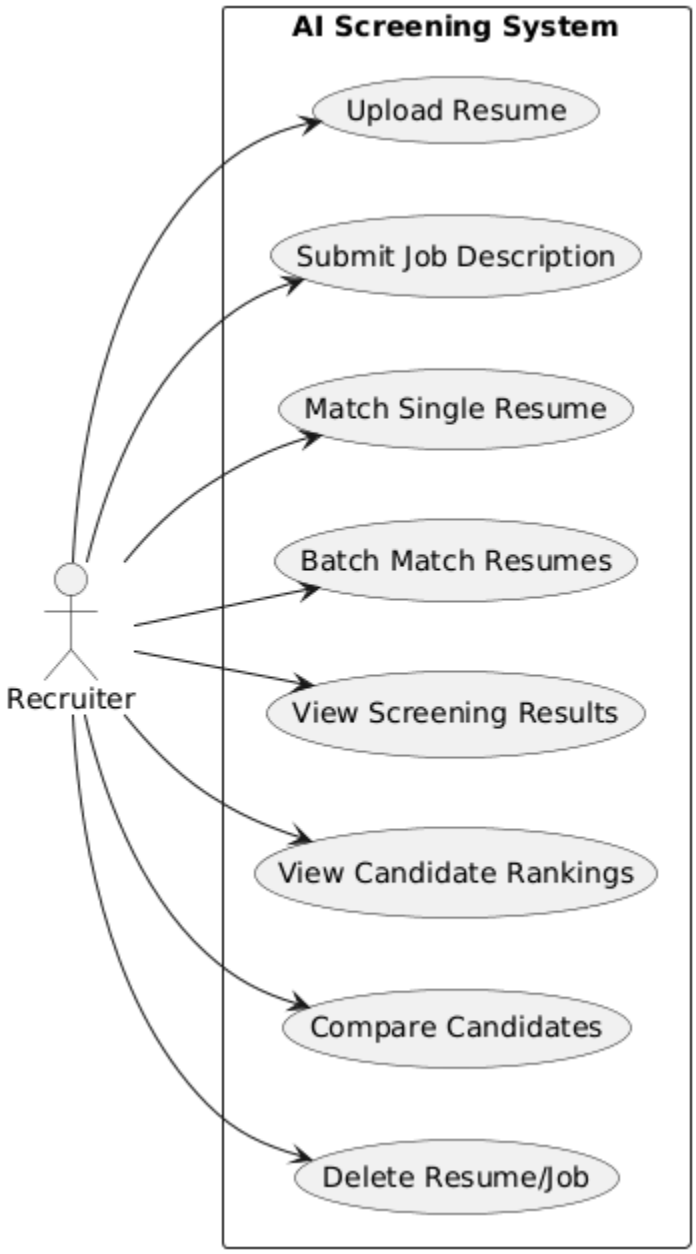

fig1: Use-case Diagram

3.3.2 Activity Diagram:

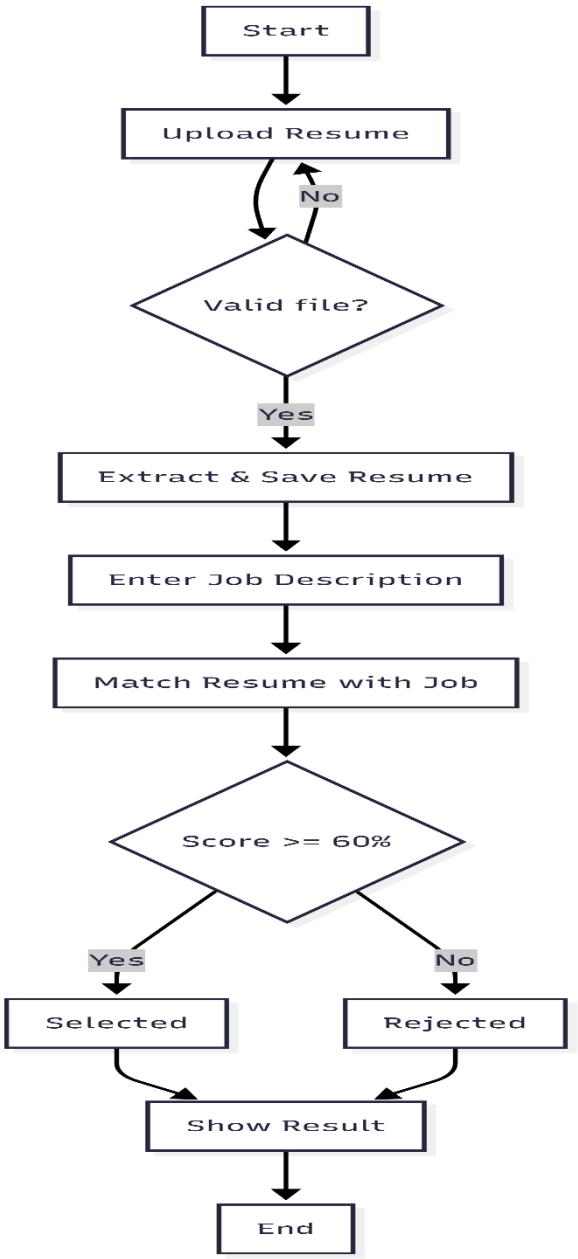

fig 2: Activity Diagram

3.3.3 Class Diagram:

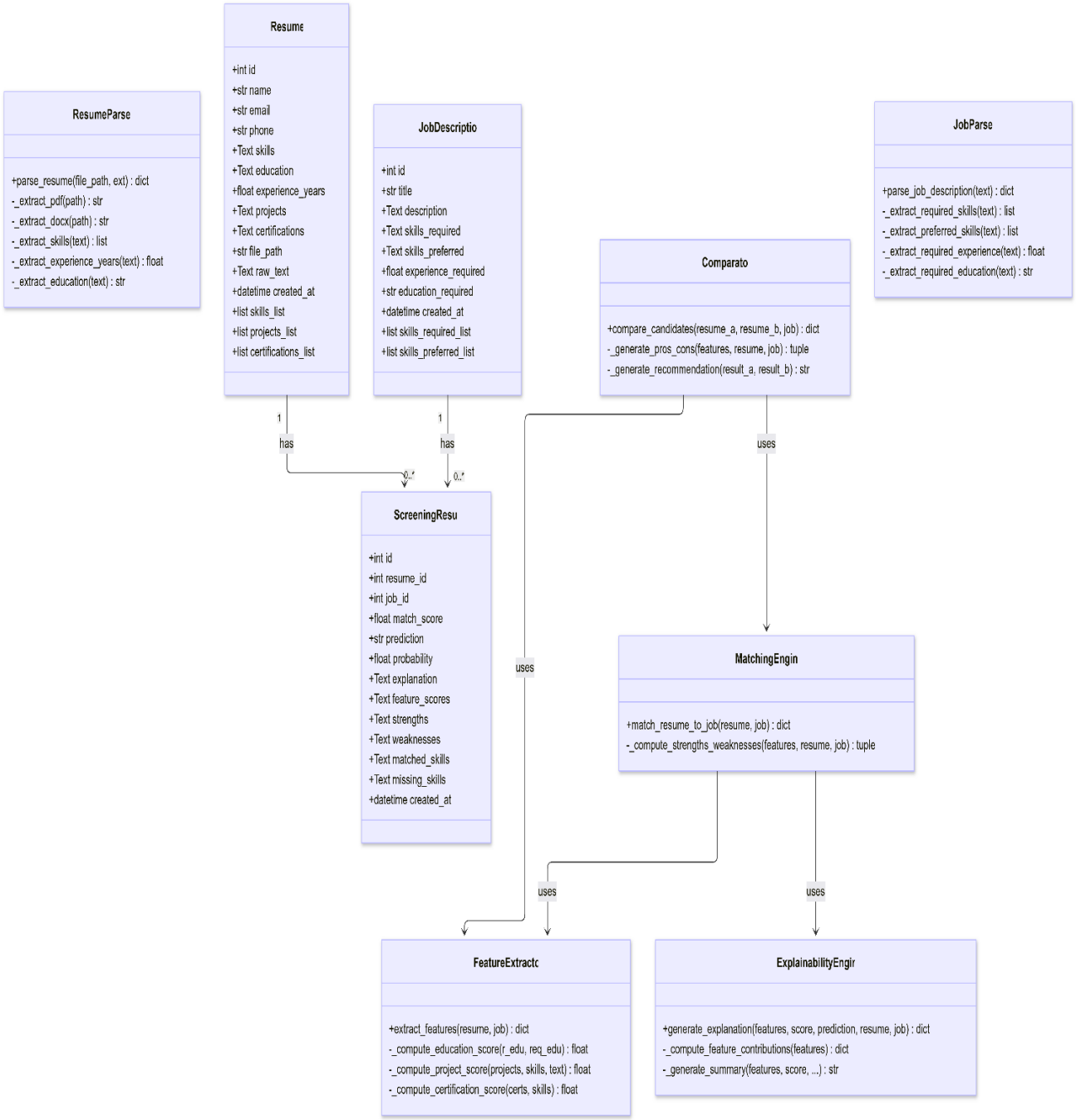

fig 3: Class Diagram

3.3.4 Sequence Diagram:

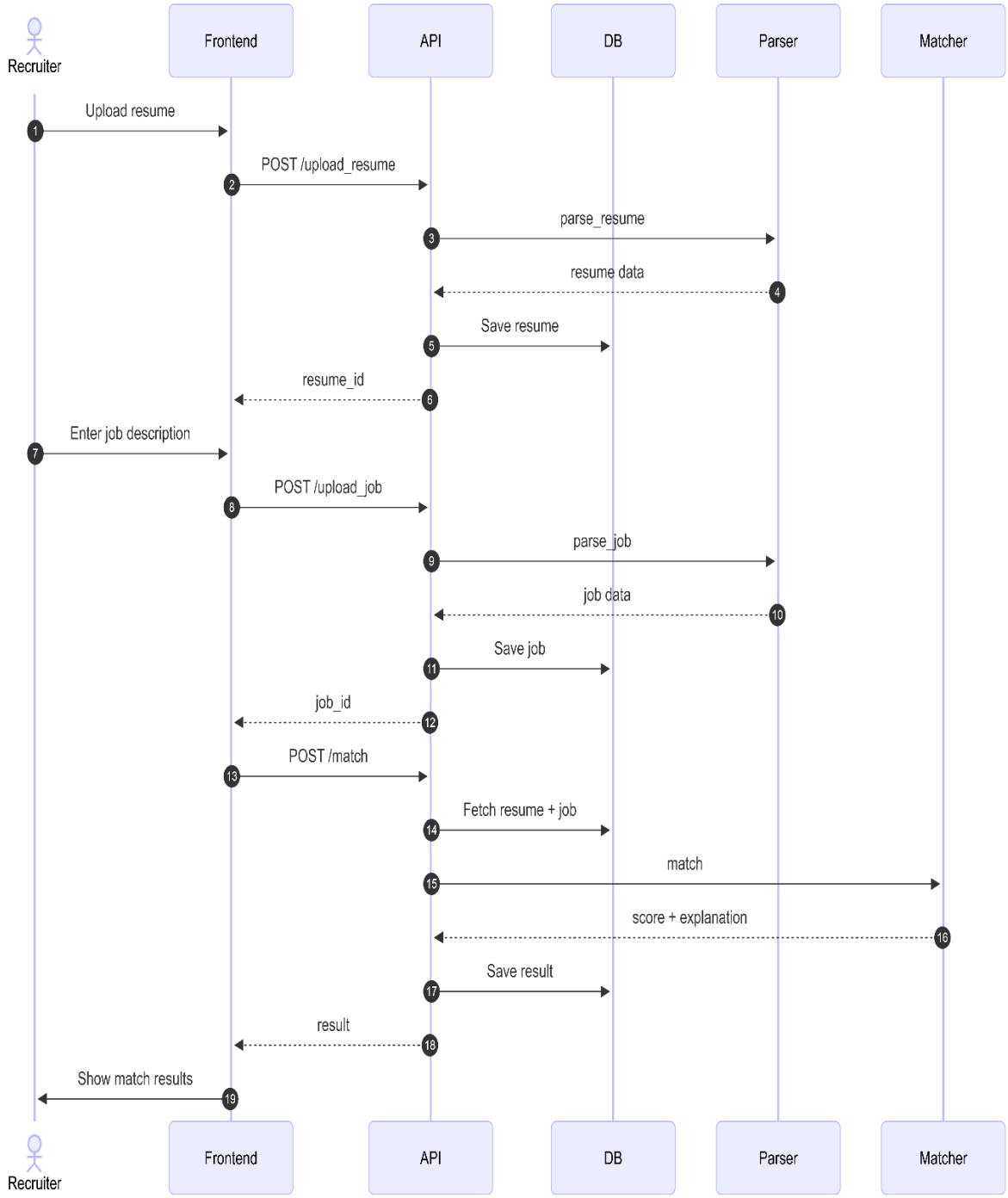

fig 4: Sequence Diagram

4 Other Non-functional Requirements

4.1 Performance Requirements

- Single match (POST /api/match_resume): response within 5 seconds for resumes ≤ 500 KB.
- Resume upload (POST /api/upload_resume): completed within 3 seconds for PDFs up to 3 pages.
- Batch matching of up to 20 resumes completes within 60 seconds (~3 seconds per resume).
- Backend idle memory ≤ 2 GB RAM; peak during matching ≤ 3 GB.

4.2 Safety and Security Requirements

- File validation: only .pdf, .docx, .txt extensions allowed; max size 10 MB; invalid files rejected with HTTP 400.
- UUID-based filenames for uploads — prevents directory traversal and filename collision attacks.
- CORS restricted to localhost:5173 and localhost:3000; all other origins rejected.
- No authentication in current scope — must add JWT/OAuth 2.0 before any production deployment.

4.3 Software Quality Attributes

- Maintainability: Six independent AI modules with a central config.py for all behaviour changes.
- Reliability: All errors handled gracefully; database sessions auto-cleaned via FastAPI dependency injection.
- Transparency: Every screening decision includes a human-readable explanation — architecturally guaranteed.
- Usability: Score gauges, feature bars, and skill tags make AI results clear to non-technical recruiters.

5 Other Requirements

5.1 Database Requirements:

- SQLite via SQLAlchemy 2.0.35; database file at database/hirelens.db; schema auto-created on startup.
- Three tables: resumes, job_descriptions, and screening_results with full CRUD and cascading delete.

5.2 Internationalization Requirements:

- All UI text, API responses, and explanations are in English only.
- UTF-8 encoding with errors='ignore' ensures non-ASCII characters do not break parsing.

5.3 Legal Requirements:

- GDPR Article 22 aligned: human-readable explanation generated for every automated screening decision.
- Data minimisation: only job-relevant fields collected; no age, gender, or nationality extracted.

5.4 Reuse Objectives:

- Six AI pipeline modules are independent Python packages — reusable outside FastAPI or React.

5.5 Development Environment Requirements:

- Backend: Python 3.10+, virtual env, pip install -r requirements.txt, then python -m spacy download en_core_web_sm.
- Frontend: Node.js 18+, npm install (in frontend/), then npm run dev.

5.6 Documentation Requirements:

- All AI pipeline functions include docstrings; critical formulas annotated with inline comments.
- README.md in project root covers setup, launch steps, API overview, and known limitations.
- This SRS document is the primary requirements reference, updated whenever system scope changes.

6 References

- IEEE Computer Society. (2011). IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications. IEEE Standards Association.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. KDD 2016, pp. 1135–1144.
- Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS 2017, 30, pp. 4765–4774.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019, pp. 3982–3992.
- FastAPI Official Documentation — https://fastapi.tiangolo.com/
- spaCy: Industrial-Strength NLP — https://spacy.io/
- pdfplumber Documentation — https://github.com/jsvine/pdfplumber
- European Commission (2016). GDPR Article 22: Automated Individual Decision-Making. Official Journal of the EU, L 119.
- React Official Documentation — https://react.dev/
- SQLAlchemy Documentation — https://www.sqlalchemy.org

SRS DOCUMENT REVIEW

CERTIFICATION

This Software Requirement Specification (SRS) Document is reviewed and certified to proceed for the project development by the Departmental Review Committee (DRC).

Date of SRS Submitted:	
Date of Review:	
Supervisor Comments:	
Supervisor Sign. & Date.	
Coordinator Sign. & Date	
HOD Sign. & Date	
Dept. Stamp	